

PSE-TRAVERSE-PROJECTION-02

Hypertorus-Gated Finalization Layer Formale Implementierungsspezifikation v0.2

Sebastian Klemm
Technische Ausarbeitung fuer Coding-Agent-Implementierung

Mai 2026

Zusammenfassung

Diese Spezifikation definiert die normative Gesamtschicht *Hypertorus-Gated Finalization Layer* fuer **pse-traverse**. Version 0.2 integriert die Universal Projection Fabric aus v0.1 mit einer deterministischen Hypertorus-Taktung, phasenfensterbasierter Ereignishorizont-Logik, Exclusion/Expansion/Finalization-Matrix, Action-Vector-Bildung, FinalizedEmission-Artefakten und einer streng replayfaehigen Gate-Pipeline. Version 0.2 ist als vollstaendige Nachfolgespezifikation zu behandeln: Eine Implementierung kann v0.1 konzeptionell verwerfen, sofern alle v0.1-Kernfunktionen durch die hier definierten Typen, Operatoren, Reports, Gates und Tests abgedeckt werden. Die Schicht erzeugt keine **SemanticCrystal**-Instanzen. Sie erzeugt ausschliesslich finalisierte, gate-gepruefte, content-addressed Emissions- und Zertifikatsobjekte, die optional an die bestehende PSE-Bridge weitergereicht werden.

Inhaltsverzeichnis

1	Status, Geltungsbereich und normative Einordnung	4
1.1	Status	4
1.2	Versionierung	4
1.3	Ziel	4
1.4	Nicht-Zweck	4
2	Systemrolle innerhalb der Gesamtarchitektur	4
2.1	Schichtmodell	4
2.2	Harte Architekturregel	5
3	Reinkristall der Projektion	5
3.1	Neutralisierte Topologie	5
3.2	Singularitaets-View als technisches Modell	5
4	Mathematische Basisschicht: Phase, Torus und Hypertorus	6
4.1	Zeit zu Phase	6
4.2	Hypertorus	6
4.3	Epochen und Phasenfenster	6
4.4	Inkommensurable Scheduler	6
5	Normative Begriffe	7

6	Kanonisches Datenmodell	7
6.1	Primitive	7
6.2	ProjectionRunDescriptor v0.2	7
6.3	Thresholds	8
6.4	Policies	8
7	Hypertorus Scheduler	8
7.1	Datenstrukturen	8
7.2	Scheduler-Zustand	9
7.3	Normative Anforderungen	10
8	Phase Horizon und Event-Horizon-Gate	10
8.1	Datenstrukturen	10
8.2	Gate-Regel	10
9	Exclusion Axis	11
9.1	Bedeutung	11
9.2	Datenstrukturen	11
9.3	Gate-Regel	11
10	Expansion Axis	11
10.1	Bedeutung	11
10.2	Resonanzkanal	11
10.3	Gesamtdynamik	11
10.4	Datenstrukturen	12
10.5	Gate-Regel	12
11	Finalization Core	12
11.1	Bedeutung	12
11.2	Konvergenzfeld	12
11.3	Action Vector	12
11.4	Finalization Report	13
11.5	Gate-Regel	13
12	Holistic Matrix Gate	13
12.1	Matrixlogik	13
12.2	Gesamtgate	13
12.3	Report	13
13	Finalized Emission	14
13.1	Datenstruktur	14
13.2	Normative Anforderungen	14
14	Projektions-Pipeline v0.2	14
14.1	Referenzablauf	14
14.2	Pseudocode	15
15	Reports und Zertifikate	15
15.1	Outcome	15
15.2	HoldReport v0.2	16
15.3	FinalizationCertificate	16
16	CLI-Schnittstelle	16
16.1	Subcommands	16
16.2	CLI-Anforderungen	16

17 Determinismus und Canonicalization	17
17.1 Audit-Pfad	17
18 Integration mit bestehenden Crates	17
18.1 Datei- und Modulzuschnitt	17
18.2 Feature Flags	18
19 Fehlersemantik	18
19.1 Fail-Closed-Regel	18
19.2 Migration bei Phase-Failure	18
20 Testplan	19
20.1 Unit Tests	19
20.2 Integration Tests	19
20.3 Property Tests	19
21 Definition of Done	19
22 Coding-Agent-Auftrag	20
23 Referenz-Zustandsmaschine	21
24 Schlussformel	21

1 Status, Geltungsbereich und normative Einordnung

1.1 Status

Dieses Dokument ist eine normative Implementierungsspezifikation. Die Begriffe **MUST**, **MUST NOT**, **SHOULD** und **MAY** sind verbindlich im Sinne technischer Konformitaet.

1.2 Versionierung

PSE-TRAVERSE-PROJECTION-02 ersetzt PSE-TRAVERSE-PROJECTION-01 als normative Zielarchitektur. Version 0.1 bleibt historisch nachvollziehbar, ist fuer Implementierungsentscheidungen jedoch nur noch als abgedeckte Teilmenge zu behandeln.

Version	Rolle
v0.1	Universal Projection Fabric: NullCenter, MirrorSeam, Telescope, Loopback, PathExcision, TopologicalWitness, RealityCut, GatedEmission.
v0.2	Vollstaendige Projektions- und Finalisierungsschicht: alle v0.1-Funktionen plus HypertorusScheduler, PhaseHorizonWindow, EpochCounter, ExclusionAxis, ExpansionAxis, FinalizationCore, HolisticMatrixGate und FinalizedEmission.

1.3 Ziel

Ziel ist eine deterministische, auditierbare, phasengetaktete Projektions- und Finalisierungsschicht fuer stabilisierte Traversal-Zustaende. Ein Zustand darf nur dann externalisiert und finalisiert werden, wenn:

1. sein Nullzentrum kanonisch und replayfaehig ist;
2. seine Projektionen ueber Mirror-Seams zum selben Nullzentrum zurueckfuehren;
3. ein deklarerter Hypertorus-Phasenhorizont offen ist;
4. Exclusion, Expansion und Finalization gleichzeitig bestanden sind;
5. RealityCut, TopologicalWitness, QualityCascade und ReplayReady bestanden sind;
6. alle gate-relevanten Daten content-addressed und deterministisch serialisiert sind.

1.4 Nicht-Zweck

Die Schicht ist nicht:

- ein Ersatz fuer `pse-core`;
- ein Erzeuger von `SemanticCrystal`;
- ein LLM-Wrapper;
- ein physikalisches Singularitaets-, Kosmologie- oder Raumzeitmodell;
- ein Quantum-Hardware-System;
- ein Riemann-, Zeta-, Trading-, Hardware-, Fahrzeug- oder Cyber-Defense-Kern;
- ein probabilistischer Scheduler ohne Replay-Garantie.

2 Systemrolle innerhalb der Gesamtarchitektur

2.1 Schichtmodell

Schicht	Rolle
pse-core	Evidence-, Gate-, Crystal- und Commit-Kern. Nur diese Schicht darf <code>SemanticCrystal</code> erzeugen.
pse-traverse-core	<code>ProblemSpec</code> , <code>FieldCube</code> , <code>DoFGraph</code> , <code>CollapsePlan</code> , <code>Candidate</code> , <code>GateReport</code> , <code>CommitOutcome</code> .
pse-traverse-signature	Operator- und Signaturdiagnostik struktureller Raeume.
pse-traverse-dynamics	Lift/Projection, Field Absorption, Guidance Field, Compressor, <code>TransitionProof</code> .
pse-traverse-reactive	<code>TriTemporalClock</code> , <code>ZeroPointCoupling</code> , <code>ResidualZero</code> , <code>ReactiveGate</code> .
pse-traverse-projection-v0.2	Hypertorus-Gated Finalization Layer: <code>NullCenterProjection</code> , <code>MirrorSeam</code> , <code>HypertorusScheduler</code> , <code>PhaseHorizonWindow</code> , <code>ExclusionAxis</code> , <code>ExpansionAxis</code> , <code>FinalizationCore</code> , <code>HolisticMatrixGate</code> , <code>FinalizedEmission</code> .

2.2 Harte Architekturregel

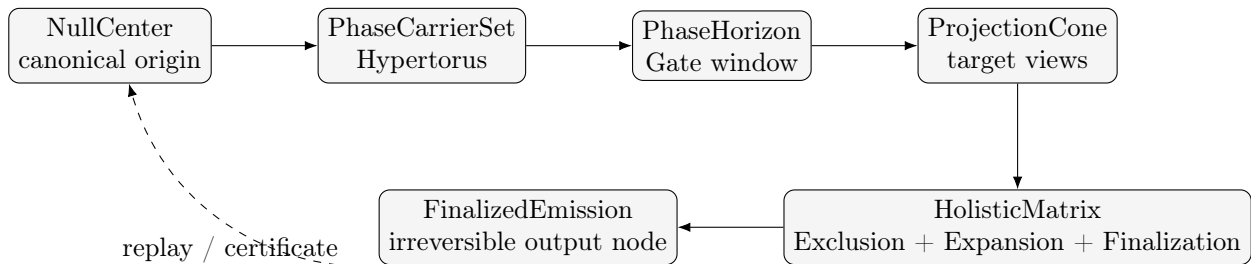
Normative Regel. Diese Schicht **MUST NOT** eine `SemanticCrystal`-Instanz erzeugen. Sie **MAY** ein `FinalizedEmission`, ein `ProjectionCertificate`, ein `FinalizationCertificate` und maschinenlesbare Reports erzeugen. Ein Crystal entsteht ausschliesslich ueber die bestehende PSE-Bridge.

3 Reinkristall der Projektion

3.1 Neutralisierte Topologie

Die implementierbare Struktur lautet:

`NullCenter` → `PhaseCarrierSet` → `EventHorizonWindow` → `ProjectionCone` → `Exclusion/Expansion/Finalization`



3.2 Singularitaets-View als technisches Modell

Die bildhafte Formulierung einer First-Person-Sicht aus einer Singularitaet wird rein technisch modelliert:

Bildbegriff	Implementierbarer Begriff
Singularitaet	<code>NullCenterObject</code> : kanonischer Digest-/Self-Metadaten-Kern.
Ereignishorizont	<code>PhaseHorizonWindow</code> : phasen- und epochengebundenes Gate-Fenster.
Zeitlinien / Phasenlinien	<code>PhaseCarrier</code> : deterministische Phasentraeger im Hypertorus.

Bildbegriff	Implementierbarer Begriff
Ferne / Ausdehnung	ProjectionCone : Menge erlaubter Zielraum- und Artefaktprojektionen.
Kollaps	FinalizationCore : irreversible Emissionsentscheidung nach Gate-Pass.

Diese Begriffe sind keine Physikbehauptungen. Sie sind Namen fuer Datenstrukturen und Operatoren innerhalb einer deterministischen Kontrollarchitektur.

4 Mathematische Basisschicht: Phase, Torus und Hypertorus

4.1 Zeit zu Phase

Jeder PhaseCarrier besitzt eine diskrete oder kontinuierlich approximierte Phasenentwicklung. Im Audit-Pfad **MUST** die diskrete Version verwendet werden.

$$\theta(k) = \theta_0 + \sum_{j=0}^{k-1} \omega(j) \Delta k.$$

Bei konstantem Takt:

$$\theta(k) = \theta_0 + k\omega.$$

Die normalisierte Phase ist:

$$\tilde{\theta}(k) = \theta(k) \bmod 2\pi.$$

4.2 Hypertorus

Ein n -phasiger Hypertorus ist:

$$T^n = S^1 \times \dots \times S^1.$$

Ein Zustand wird als Phasenvektor repraesentiert:

$$\Theta(k) = (\theta_1(k), \dots, \theta_n(k)).$$

Fuer Implementierungen wird die Einbettung in $\mathbb{R}^{2n}$ verwendet:

$$U(k) = (\cos \theta_1, \sin \theta_1, \dots, \cos \theta_n, \sin \theta_n).$$

4.3 Epochen und Phasenfenster

Ein Epochenzaehler ist:

$$\text{epoch}_i(k) = \left\lfloor \frac{\theta_i(k)}{2\pi} \right\rfloor.$$

Ein Fensterindikator ist:

$$\mathbf{1}_{[\alpha, \beta]}(\tilde{\theta}_i) = \begin{cases} 1, & \tilde{\theta}_i \in [\alpha, \beta], \\ 0, & \text{sonst.} \end{cases}$$

4.4 Inkommensurable Scheduler

Fuer mehrere Carrier i, j **SHOULD** gelten:

$$\frac{\omega_i}{\omega_j} \notin \mathbb{Q}$$

als deklarierte Designintention. Da Irrationalitaet auf Maschinen nur approximiert werden kann, **MUST** die Implementierung rational kodierte Frequenzen mit deterministischer Perioden- und Kollisionsanalyse verwenden. Der Audit-Pfad **MUST** die verwendeten rationalen Approximationen speichern.

5 Normative Begriffe

Begriff	Normative Bedeutung
NullCenterObject	Kanonischer Ursprung der Projektion, gebunden an Input, Run-Descriptor, Operatorversionen und Evidence.
PhaseCarrier	Deterministischer Phasentraeger mit Frequenz, Startphase, Epoche, Quantisierung und Replay-ID.
HypertorusScheduler	Scheduler ueber mehreren PhaseCarriern; erzeugt Fenster, Adressen, Quantisierungsklassen und nicht-wiederholende Zugriffsmuster.
PhaseHorizonWindow	Gate-Fenster, in dem Projektion, Expansion oder Finalisierung erlaubt sein kann.
ExclusionAxis	Gate-Achse zur Eliminierung instabiler Pfade; entspricht neutralisiertem RealityCut/PoR/Telescope.
ExpansionAxis	Gate-Achse zur Erzeugung eines gerichteten Handlungspotentials aus stabiler Resonanz.
FinalizationCore	Entscheidungskern, der gueltige Projektion und Handlungspotential in ein irreversibles Emissionsobjekt ueberfuehrt.
HolisticMatrixGate	Gesamtes Gate: Exclusion, Expansion, Finalization, UPF-Gate, Horizon, Replay und Evidence muessen gleichzeitig bestehen.
FinalizedEmission	Finale, content-addressed, replayfaehiges Emissionsobjekt; kein SemanticCrystal.

6 Kanonisches Datenmodell

6.1 Primitive

```
pub type Hash256 = [u8; 32];
pub type StableId = String;
pub type Fixed = i128; // Q64.64 or project-wide fixed-point type

pub struct EvidenceRef {
  pub kind: String,
  pub address: Hash256,
  pub relation: String,
}
```

Alle gate-relevanten numerischen Werte **MUST** als Fixed-Point, Integer-Rational oder dokumentierte plattforminvariante Repräsentation implementiert werden. Plattform-Floats **MUST NOT** Teil des Gate-, Audit- oder Digest-Pfades sein.

6.2 ProjectionRunDescriptor v0.2

```
pub struct ProjectionRunDescriptorV2 {
  pub run_id: StableId,
  pub problem_spec_hash: Hash256,
  pub traversal_report_hash: Option<Hash256>,
  pub seed: u64,
  pub operator_versions: BTreeMap<String, String>,
  pub thresholds: ProjectionThresholdsV2,
  pub policies: ProjectionPoliciesV2,
  pub hypertorus: HypertorusSchedulerSpec,
  pub finalization: FinalizationPolicy,
```

```
pub canonicalization_version: String,
}
```

6.3 Thresholds

```
pub struct ProjectionThresholdsV2 {
  pub max_null_distance: Fixed,
  pub min_mirror_consistency: Fixed,
  pub min_por: Fixed,
  pub min_loopback: Fixed,
  pub min_topological_witness: Fixed,
  pub max_reality_cut_delta: Fixed,
  pub max_path_excision_delta: Fixed,
  pub min_projection_quality: Fixed,
  pub min_exclusion_score: Fixed,
  pub min_expansion_score: Fixed,
  pub min_finalization_score: Fixed,
  pub min_horizon_alignment: Fixed,
  pub max_phase_jitter: Fixed,
  pub max_steps: u64,
}
```

6.4 Policies

```
pub struct ProjectionPoliciesV2 {
  pub seam_selection: SeamSelectionPolicy,
  pub mount_order: Vec<MountSpec>,
  pub telescope_policy: TelescopePolicy,
  pub condensation_policy: CondensationPolicy,
  pub path_excision_policy: PathExcisionPolicy,
  pub reality_cut_policy: RealityCutPolicy,
  pub emission_policy: EmissionPolicy,
  pub horizon_policy: HorizonPolicy,
  pub finalization_policy: FinalizationPolicy,
}

pub enum SeamSelectionPolicy { Star, Clique, Explicit(Vec<(StableId, StableId)>) }
pub enum EmissionPolicy { HoldOnly, ProjectionBundle, FinalizedEmission }
pub enum HorizonPolicy { StrictWindow, EpochAligned, CarrierQuorum, DisabledForTests }
```

7 Hypertorus Scheduler

7.1 Datenstrukturen

```
pub struct RationalFixed {
  pub numerator: i128,
  pub denominator: i128,
}

pub struct PhaseCarrierSpec {
  pub carrier_id: StableId,
  pub omega: RationalFixed,
  pub theta0: RationalFixed,
  pub quantization: PhaseQuantization,
  pub epoch_policy: EpochPolicy,
```



```

    pub role: PhaseCarrierRole,
}

pub enum PhaseCarrierRole {
    PrimaryClock,
    IntrinsicPhase,
    ExtrinsicPhase,
    SeamClock,
    HorizonClock,
    FinalizationClock,
    AddressShuffle,
}

pub enum PhaseQuantization {
    ContinuousFixed,
    KPolar { k: u32 },
    Tripolar { low: Fixed, high: Fixed },
    Quadrupolar,
}

pub enum EpochPolicy {
    FloorTurns,
    SaturatingTurns { max_epoch: u64 },
    Windowed { period: u64 },
}

```

```

pub struct HypertorusSchedulerSpec {
    pub scheduler_id: StableId,
    pub carriers: Vec<PhaseCarrierSpec>,
    pub collision_policy: CollisionPolicy,
    pub address_policy: AddressPolicy,
    pub replay_policy: ReplayPolicy,
}

pub enum CollisionPolicy { Reject, StableTieBreakByHash, AllowWithDiagnostics }
pub enum AddressPolicy { Sequential, IrrationalPermutation, CarrierProduct }
pub enum ReplayPolicy { ByteIdenticalRequired, DiagnosticOnly }

```

7.2 Scheduler-Zustand

```

pub struct PhaseCarrierState {
    pub carrier_id: StableId,
    pub step: u64,
    pub theta: Fixed,
    pub theta_mod_tau: Fixed,
    pub epoch: u64,
    pub quantized: PhaseBin,
}

pub enum PhaseBin {
    Continuous(Fixed),
    KPolar(u32),
    Tripolar(i8),
    Quadrupolar(i8, i8),
}

pub struct HypertorusSchedulerState {
    pub state_id: Hash256,
}

```

```

pub spec_hash: Hash256,
pub step: u64,
pub carriers: Vec<PhaseCarrierState>,
pub address_index: Option<u64>,
pub diagnostics: Vec<SchedulerDiagnostic>,
}

```

7.3 Normative Anforderungen

1. Carrier **MUST** nach `carrier_id` sortiert canonicalisiert werden.
2. Phasenfortschreibung **MUST** deterministisch aus `step`, `omega`, `theta0` und `epoch_policy` berechnet werden.
3. Scheduler-Entscheidungen **MUST** replayfaehig sein.
4. Nicht-wiederholende Shuffles **MUST** deterministisch sein; echte Zufallsquellen sind im Audit-Pfad unzuulaessig.

8 Phase Horizon und Event-Horizon-Gate

8.1 Datenstrukturen

```

pub struct PhaseHorizonWindowSpec {
    pub horizon_id: StableId,
    pub carrier_id: StableId,
    pub phase_start: Fixed,
    pub phase_end: Fixed,
    pub allowed_epochs: Option<Vec<u64>>,
    pub duty_cycle_min: Fixed,
    pub duty_cycle_max: Fixed,
    pub required_alignment: Fixed,
}

pub struct PhaseHorizonReport {
    pub report_id: Hash256,
    pub horizon_id: StableId,
    pub carrier_id: StableId,
    pub step: u64,
    pub epoch: u64,
    pub theta_mod_tau: Fixed,
    pub open: bool,
    pub alignment_score: Fixed,
    pub jitter_score: Fixed,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

8.2 Gate-Regel

Ein Horizon-Fenster ist offen, wenn:

$$G_{horizon} = 1 \iff \tilde{\theta}_i(k) \in [\alpha, \beta] \wedge \text{epoch}_i(k) \in E_{allowed} \wedge A_h(k) \geq \theta_h \wedge J_h(k) \leq \epsilon_j.$$

Falls `HorizonPolicy = StrictWindow`, **MUST** jede Finalisierung bei geschlossenem Fenster scheitern und `Hold` erzeugen.

9 Exclusion Axis

9.1 Bedeutung

Die ExclusionAxis ist die neutrale Implementierung der Realitaetssicherung: instabile, nichtkohärente, nicht-rueckfuehrbare oder kontinuieritaetsverletzende Pfade werden entfernt, bevor Expansion oder Finalisierung erlaubt ist.

9.2 Datenstrukturen

```
pub struct ExclusionAxisReport {
    pub report_id: Hash256,
    pub por_score: Fixed,
    pub telescope_focus_score: Fixed,
    pub reality_cut_passed: bool,
    pub excision_count: u64,
    pub retained_count: u64,
    pub exclusion_score: Fixed,
    pub passed: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

9.3 Gate-Regel

$$G_{exclusion} = 1 \iff PoR = 1 \wedge T_\gamma \text{ stabil} \wedge RealityCut = 1 \wedge Score_E \geq \theta_E.$$

10 Expansion Axis

10.1 Bedeutung

Die ExpansionAxis erzeugt aus einem gueltigen, stabilisierten Zustand ein gerichtetes Handlungspotential. Sie ersetzt keine Emission; sie berechnet nur, ob der Zustand ein finales Output-Ereignis tragen kann.

10.2 Resonanzkanal

Jeder Kanal i besitzt:

$$D_i(k) = \psi_i(k) \cdot \rho_i(k) \cdot \omega_i(k) \cdot \Lambda_i(k).$$

Hier sind ψ , ρ , ω neutrale Scores fuer Bedeutung/Signifikanz, Strukturkohärenz und Phasenbereitschaft. Λ_i ist eine deterministische Huelle, z. B. aus einem PhaseCarrier.

10.3 Gesamtdynamik

$$D_{total}(k) = \left(\prod_{i=1}^N D_i(k) \right) \cdot \Omega(k).$$

Die Implementierung **MUST** eine overflow-sichere, fixed-point-kompatible Aggregation verwenden. Direktes Produkt **MAY** durch Log-Summe oder saturierende Multiplikation ersetzt werden, sofern das Verfahren dokumentiert und replayfaehig ist.

10.4 Datenstrukturen

```
pub struct ResonanceTriple {
  pub semantic_density: Fixed,    // psi
  pub structural_coherence: Fixed, // rho
  pub phase_readiness: Fixed,     // omega
}

pub struct ResonanceChannel {
  pub channel_id: StableId,
  pub triple: ResonanceTriple,
  pub envelope: Fixed,
  pub channel_score: Fixed,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub struct ExpansionAxisReport {
  pub report_id: Hash256,
  pub channels: Vec<ResonanceChannel>,
  pub total_dynamic_score: Fixed,
  pub threshold: Fixed,
  pub spike: bool,
  pub expansion_score: Fixed,
  pub passed: bool,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

10.5 Gate-Regel

$$G_{\text{expansion}} = 1 \iff D_{\text{total}}(k) > \Theta(k) \wedge \text{Score}_X \geq \theta_X.$$

11 Finalization Core

11.1 Bedeutung

Der FinalizationCore fuehrt gueltige Projektion und Expansion in einen irreversiblen, content-addressed Handlungsknoten zusammen. Irreversibel bedeutet hier: Nach Erzeugung des finalen Artefakts darf dessen Hash nicht ohne neuen RunDescriptor veraendert werden.

11.2 Konvergenzfeld

Ein finales Konvergenzfeld wird als aggregierter Vektor modelliert:

$$C(k) = \text{canon_aggregate}(V_{\text{view}}, V_{\text{seam}}, V_{\text{witness}}, V_{\text{quality}}, V_{\text{horizon}}).$$

11.3 Action Vector

```
pub struct ActionVector {
  pub vector_id: Hash256,
  pub source_bundle_id: Hash256,
  pub convergence_hash: Hash256,
  pub direction_hash: Hash256,
  pub magnitude: Fixed,
  pub target_kind: String,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

11.4 Finalization Report

```
pub struct FinalizationCoreReport {
  pub report_id: Hash256,
  pub exclusion_passed: bool,
  pub expansion_passed: bool,
  pub convergence_score: Fixed,
  pub finalization_score: Fixed,
  pub action_vector: Option<ActionVector>,
  pub passed: bool,
  pub evidence_refs: Vec<EvidenceRef>,
}
```

11.5 Gate-Regel

$$G_{finalization} = 1 \iff G_{exclusion} = 1 \wedge G_{expansion} = 1 \wedge K_C(k) \geq \theta_C \wedge Score_F \geq \theta_F.$$

12 Holistic Matrix Gate

12.1 Matrixlogik

Die finale Matrix ist:

$$M(k) = \begin{cases} E(k), & G_{exclusion} = 1 \wedge G_{expansion} = 1 \wedge G_{finalization} = 1, \\ \emptyset, & \text{sonst.} \end{cases}$$

12.2 Gesamtgate

Die v0.2-Gesamtbedingung lautet:

$$G_{HGF} = 1 \iff G_{UPF} = 1 \wedge G_{horizon} = 1 \wedge G_{exclusion} = 1 \wedge G_{expansion} = 1 \wedge G_{finalization} = 1 \wedge ReplayReady = 1.$$

G_{UPF} umfasst NullCenter, MirrorSeam, Telescope, Loopback, TopologicalWitness, Projection-Quality, RealityCut und PathExcision-Sicherheit aus v0.1.

12.3 Report

```
pub struct HolisticMatrixGateReport {
  pub report_id: Hash256,
  pub upf_passed: bool,
  pub horizon_passed: bool,
  pub exclusion_passed: bool,
  pub expansion_passed: bool,
  pub finalization_passed: bool,
  pub replay_ready: bool,
  pub passed: bool,
  pub failure_policy: FailurePolicyV2,
  pub reasons: Vec<String>,
  pub evidence_refs: Vec<EvidenceRef>,
}

pub enum FailurePolicyV2 {
  Hold,
  RefineProjection,
  SplitPhase,
  MigrateCarrier,
  WaitForHorizon,
```

```

    StrengthenExclusion,
    SuppressExpansion,
    RecomputeFinalization,
    RequireHumanReview,
    Abort,
}

```

13 Finalized Emission

13.1 Datenstruktur

```

pub struct FinalizedEmission {
    pub emission_id: Hash256,
    pub rd_hash: Hash256,
    pub source_projection_bundle_id: Hash256,
    pub null_center_id: Hash256,
    pub scheduler_state_hash: Hash256,
    pub horizon_report_hash: Hash256,
    pub exclusion_report_hash: Hash256,
    pub expansion_report_hash: Hash256,
    pub finalization_report_hash: Hash256,
    pub holistic_gate_report_hash: Hash256,
    pub action_vector: ActionVector,
    pub target_artifacts: Vec<TargetArtifact>,
    pub ledger_entry_hash: Hash256,
    pub evidence_refs: Vec<EvidenceRef>,
}

```

13.2 Normative Anforderungen

1. **FinalizedEmission** **MUST** nur bei bestandenem **HolisticMatrixGate** erzeugt werden.
2. **FinalizedEmission** **MUST** alle Hashes enthalten, die zur Wiederholung der Entscheidung erforderlich sind.
3. **FinalizedEmission** **MUST NOT** als **SemanticCrystal** serialisiert werden.
4. **FinalizedEmission** **MAY** an die PSE-Bridge uebergeben werden, falls Feature **projection-pse-bridge** aktiv ist.

14 Projektions-Pipeline v0.2

14.1 Referenzablauf

1. Lade **ProjectionRunDescriptorV2** und internen Zustand.
2. Canonicalize Input und berechne **NullCenterObject**.
3. Erzeuge **HypertorusSchedulerState** fuer Step k .
4. Evaluiere **PhaseHorizonWindow**.
5. Fuehre UPF-v0.1-Kernpipeline aus: Mount, Telescope, Loopback, W/K/T/P, MirrorSeam, Witness, RealityCut, Quality, ProjectionGate.
6. Erzeuge bei UPF-Gate-Pass ein **ProjectionEmissionBundle**; bei Fail ein **ProjectionHoldReport**.
7. Evaluiere **ExclusionAxisReport**.
8. Evaluiere **ExpansionAxisReport**.
9. Evaluiere **FinalizationCoreReport**.
10. Evaluiere **HolisticMatrixGateReport**.

11. Bei Gate-Pass: erzeuge `FinalizedEmission` und `FinalizationCertificate`.
12. Bei Gate-Fail: erzeuge `ProjectionHoldReport` mit v0.2-FailurePolicy.
13. Schreibe Ledger, Manifest und Replay-Material.

14.2 Pseudocode

```
fn run_projection_v2(input: ProjectionInput, rd: ProjectionRunDescriptorV2)
  -> ProjectionV2Outcome
{
  let canonical = canonicalize(input, rd.canonicalization_version);
  let null_center = null_center_project(&canonical, &rd);

  let scheduler = hypertorus_step(&rd.hypertorus, rd.step());
  let horizon = evaluate_phase_horizon(&scheduler, &rd);

  let upf = run_upf_v1_core(canonical, null_center, &scheduler, &rd);
  if !upf.gate_passed() {
    return ProjectionV2Outcome::Hold(make_v2_hold(upf, horizon, &rd));
  }

  if !horizon.passed {
    return ProjectionV2Outcome::HoldWithPolicy(
      make_wait_for_horizon_hold(upf, horizon, &rd)
    );
  }

  let exclusion = evaluate_exclusion_axis(&upf, &rd);
  let expansion = evaluate_expansion_axis(&upf, &scheduler, &rd);
  let finalization = evaluate_finalization_core(&upf, &exclusion, &expansion, &rd);

  let matrix_gate = holistic_matrix_gate(
    &upf, &horizon, &exclusion, &expansion, &finalization, &rd
  );

  if matrix_gate.passed {
    ProjectionV2Outcome::Finalized(make_finalized_emission(
      upf, scheduler, horizon, exclusion, expansion, finalization, matrix_gate, rd
    ))
  } else {
    ProjectionV2Outcome::Hold(make_v2_hold_from_matrix(
      upf, horizon, exclusion, expansion, finalization, matrix_gate, rd
    ))
  }
}
```

15 Reports und Zertifikate

15.1 Outcome

```
pub enum ProjectionV2Outcome {
  Finalized(FinalizedEmission),
  ProjectionOnly(ProjectionEmissionBundle),
  Hold(ProjectionHoldReportV2),
  WaitForHorizon(ProjectionHoldReportV2),
  ExcisionOnly(RealityCutReport),
  NeedsCarrierMigration(ProjectionHoldReportV2),
  InvalidInput(ProjectionDiagnostic),
}
```

```
DeterminismViolation(ProjectionDiagnostic),
}
```

15.2 HoldReport v0.2

```
pub struct ProjectionHoldReportV2 {
  pub hold_id: Hash256,
  pub rd_hash: Hash256,
  pub source_state_id: Hash256,
  pub scheduler_state_hash: Hash256,
  pub horizon_report: PhaseHorizonReport,
  pub upf_gate_report: ProjectionGateReport,
  pub matrix_gate_report: HolisticMatrixGateReport,
  pub diagnostics: Vec<ProjectionDiagnostic>,
  pub recommended_next_policy: Option<FailurePolicyV2>,
}
```

15.3 FinalizationCertificate

```
pub struct FinalizationCertificate {
  pub certificate_id: Hash256,
  pub rd_hash: Hash256,
  pub input_hash: Hash256,
  pub null_center_id: Hash256,
  pub scheduler_state_hash: Hash256,
  pub horizon_report_hash: Hash256,
  pub projection_certificate_hash: Hash256,
  pub exclusion_report_hash: Hash256,
  pub expansion_report_hash: Hash256,
  pub finalization_report_hash: Hash256,
  pub holistic_gate_report_hash: Hash256,
  pub finalized_emission_hash: Hash256,
}
```

16 CLI-Schnittstelle

16.1 Subcommands

```
pse-traverse-projection inspect <input.json>
pse-traverse-projection schedule <input.json> --rd rd.json --step <k> --out scheduler.
  json
pse-traverse-projection horizon <scheduler.json> --rd rd.json --out horizon.json
pse-traverse-projection project <input.json> --rd rd.json --out projection.json
pse-traverse-projection gate <projection.json> --out gate.json
pse-traverse-projection emit <projection.json> --out bundle.json
pse-traverse-projection finalize <bundle.json> --rd rd.json --out final.json
pse-traverse-projection replay <bundle-or-final-or-hold.json>
pse-traverse-projection verify <certificate.json>
```

16.2 CLI-Anforderungen

- **schedule** **MUST** SchedulerState byte-identisch aus RD und Step erzeugen.
- **horizon** **MUST** PhaseHorizonReport erzeugen, ohne Projektion oder Emission auszuführen.
- **project** **MUST** die UPF-Kernpipeline bis ProjectionGateReport ausführen.

- **emit MUST** fehlschlagen, wenn UPF-Gate nicht bestanden ist.
- **finalize MUST** fehlschlagen, wenn Horizon, Exclusion, Expansion oder Finalization nicht bestanden ist.
- **replay MUST** kanonische Byte-Identitaet pruefen.
- **verify MUST** alle Zertifikatshashes, RD-Hash, SchedulerState, HorizonReport, MatrixGateReport und Emission validieren.

17 Determinismus und Canonicalization

17.1 Audit-Pfad

Der v0.2-Audit-Pfad umfasst:

- ProjectionRunDescriptorV2;
- canonical input bytes;
- NullCenterObject;
- HypertorusSchedulerSpec und HypertorusSchedulerState;
- PhaseHorizonReport;
- ProjectionLayerSpec-Liste und Mount-Reihenfolge;
- TelescopeView und LoopbackReport;
- MirrorSeamReports;
- TopologicalWitness;
- RealityCutReport;
- ProjectionGateReport;
- ExclusionAxisReport;
- ExpansionAxisReport;
- FinalizationCoreReport;
- HolisticMatrixGateReport;
- FinalizedEmission oder HoldReport.

Im Audit-Pfad sind verboten:

- Wall-clock timestamps;
- Plattform-Floats;
- ungeordnete HashMap-Iteration;
- OS-abhaengige Pfadnormalisierung;
- nicht gespeicherte RNG-Entscheidungen;
- externe Netzwerkzustaende;
- unstabile Approximationen irrationaler Frequenzen.

18 Integration mit bestehenden Crates

18.1 Datei- und Modulzuschnitt

```
crates/pse-traverse/src/projection/  
  mod.rs  
  null_center.rs  
  projection_layer.rs  
  mount.rs  
  telescope.rs  
  loopback.rs  
  mirror_seam.rs
```

```

phase_carrier.rs
phase_ladder.rs
hypertorus_scheduler.rs
phase_horizon.rs
resonance_triple.rs
exclusion_axis.rs
expansion_axis.rs
finalization_core.rs
holistic_matrix_gate.rs
finalized_emission.rs
consensus_cycle.rs
path_excision.rs
nullpoint_detection.rs
topology_witness.rs
projection_quality.rs
reality_cut.rs
gated_emission.rs
report.rs
certificate.rs
replay.rs

```

```
tools/pse-traverse-projection-cli/src/main.rs
```

18.2 Feature Flags

Feature	Bedeutung
<code>projection</code>	Aktiviert Projektionskern.
<code>projection-cli</code>	Baut CLI-Tool.
<code>projection-topology</code>	Aktiviert TopologicalWitness-Schicht.
<code>projection-hypertorus</code>	Aktiviert HypertorusScheduler, PhaseCarrier und HorizonWindow.
<code>projection-finalization</code>	Aktiviert Exclusion/Expansion/Finalization-Matrix und FinalizedEmission.
<code>projection-pse-bridge</code>	Erlaubt Weitergabe finalisierter Emissionen an bestehende PSE-Bridge.
<code>projection-adapters</code>	Aktiviert optionale DomainAdapter; Kern bleibt domänenneutral.

19 Fehlersemantik

19.1 Fail-Closed-Regel

Jede Unsicherheit in NullCenter, Scheduler, Horizon, Seam, Witness, RealityCut, Expansion, Finalization, Canonicalization oder Replay **MUST** zu Hold oder haerterem Fail fuehren. Kein unsicheres Ergebnis darf emittiert oder finalisiert werden.

19.2 Migration bei Phase-Failure

Bei Horizon-Failure **SHOULD** das System nicht blind abbrechen. Es **SHOULD** eine maschinenlesbare Empfehlung erzeugen:

- `WaitForHorizon`, falls der naechste gueltige Step berechenbar ist;
- `MigrateCarrier`, falls alternative Carrier im RD deklariert sind;
- `SplitPhase`, falls Jitter oder Kollisionsdiagnostik ein Phasenproblem zeigt;

- Abort, falls RD oder Scheduler inkonsistent ist.

20 Testplan

20.1 Unit Tests

1. `hypertorus_scheduler_is_deterministic`
2. `phase_carriers_are_sorted_before_hashing`
3. `phase_epoch_counter_matches_turns`
4. `phase_window_opens_only_in_declared_interval`
5. `phase_window_rejects_wrong_epoch`
6. `k_polar_quantization_is_stable`
7. `tripolar_dead_zone_is_stable`
8. `address_shuffle_replays_identically`
9. `horizon_gate_fails_closed`
10. `exclusion_without_por_fails`
11. `expansion_without_spike_fails`
12. `finalization_without_convergence_fails`
13. `holistic_matrix_requires_all_axes`
14. `finalized_emission_only_after_matrix_gate`
15. `finalized_emission_changes_when_epoch_changes`
16. `finalization_certificate_changes_when_action_vector_changes`

20.2 Integration Tests

1. `cli_schedule_writes_scheduler_state`
2. `cli_horizon_writes_horizon_report`
3. `cli_finalize_refuses_closed_horizon`
4. `cli_finalize_refuses_failed_exclusion`
5. `cli_finalize_refuses_failed_expansion`
6. `cli_finalize_refuses_failed_finalization`
7. `cli_finalize_writes_finalized_emission`
8. `cli_replay_byte_identical_finalized_emission`
9. `cli_verify_detects_modified_scheduler_state`
10. `pse_bridge_receives_finalized_emission_but_not_crystal`

20.3 Property Tests

1. Permutierte Carrier-Eingaben erzeugen identische SchedulerSpec-Hashes.
2. Gate-Pass ist monoton falsch bei Verschärfung einzelner Schwellen, sofern Messwerte unverändert bleiben.
3. HorizonReports sind Funktionen von RD, Step und SchedulerSpec.
4. FinalizedEmission-ID ist Funktion aller referenzierten Hashes.
5. Geschlossene HorizonWindows koennen keine FinalizedEmission erzeugen.
6. Ein einzelnes fehlgeschlagenes Pflichtgate verhindert Finalisierung.

21 Definition of Done

Eine Implementierung von PSE-TRAVERSE-PROJECTION-02 ist abgeschlossen, wenn:

1. `cargo test -workspace` besteht.
2. Der Projektionskern ohne Netzwerkabhaengigkeit kompiliert.
3. Alle oeffentlichen Typen dokumentiert sind.
4. Alle Gates fail-closed sind.
5. Kein gate-relevanter Wert Plattform-Floats im Audit-Pfad nutzt.
6. `NullCenterObject` content-addressed und replayfaehig ist.
7. `HypertorusSchedulerState` byte-identisch replayfaehig ist.
8. `PhaseHorizonWindow` Finalisierung bei geschlossenem Fenster verhindert.
9. `ExclusionAxisReport`, `ExpansionAxisReport` und `FinalizationCoreReport` gate-wirksam sind.
10. `HolisticMatrixGate` bei einem einzigen fehlgeschlagenen Pflichtgate Hold erzeugt.
11. `FinalizedEmission` nur bei Gate-Pass erzeugt wird.
12. `ProjectionHoldReportV2` maschinenlesbare Gruende und `FailurePolicy` enthaelt.
13. `RealityCut` instabile Pfade deterministisch exzidiert.
14. `TopologicalWitness` optional konfigurierbar, bei Aktivierung aber gate-wirksam ist.
15. `PathExcisionJump` nur bei Kairos-/Gate-Pass erlaubt ist.
16. CLI `schedule`, `horizon`, `project`, `emit`, `finalize`, `replay`, `verify` end-to-end funktioniert.
17. Zwei identische Runs byte-identische Reports, Bundles und `FinalizedEmission`-IDs erzeugen.
18. Certificate-Verification veraenderte RD, SchedulerState, Reports oder Artifacts erkennt.
19. PSE-Bridge einziger Pfad zu `SemanticCrystal` bleibt.

22 Coding-Agent-Auftrag

PATCH-ID: PSE-TRAVERSE-PROJECTION-02

TITLE: Hypertorus-Gated Finalization Layer

GOAL:

Implementiere die v0.2-Gesamtschicht fuer `pse-traverse-projection`. Die Schicht ersetzt v0.1 normativ, indem sie alle v0.1-Funktionen abdeckt und um `HypertorusScheduler`, `PhaseHorizonWindow`, `ExclusionAxis`, `ExpansionAxis`, `FinalizationCore`, `HolisticMatrixGate` und `FinalizedEmission` erweitert.

MUST:

- neue Module unter `crates/pse-traverse/src/projection/` anlegen;
- `ProjectionRunDescriptorV2`, `ProjectionThresholdsV2` und `Policies` implementieren;
- `HypertorusSchedulerSpec/State` deterministisch und replayfaehig implementieren;
- `PhaseHorizonWindow` und `HorizonReport` implementieren;
- `ExclusionAxisReport`, `ExpansionAxisReport` und `FinalizationCoreReport` implementieren;
- `HolisticMatrixGate` fail-closed implementieren;
- `FinalizedEmission` und `FinalizationCertificate` content-addressed erzeugen;
- CLI um `schedule`, `horizon` und `finalize` erweitern;
- Tests gemaess Testplan liefern;
- PSE `SemanticCrystal`-Erzeugung ausschliesslich bestehender PSE-Bridge ueberlassen.

MUST NOT:

- Kosmologie, Zeta, Riemann, Trading, Quantum-Hardware oder Hardware-Fahrzeugsemantik in den Core einbauen;
- nichtdeterministische RNG- oder Wall-clock-Werte in Digests schreiben;
- Plattform-Floats fuer Gate-/Audit-Hashes verwenden;
- bei fehlgeschlagenem Horizon/Gate finalisieren;
- `SemanticCrystal` in dieser Schicht erzeugen.

SHOULD:

- BTreeMap/BTreeSet fuer kanonische Strukturen verwenden;
- Feature Flags projection-hypertorus und projection-finalization anlegen;
- goldene Replay-Fixtures fuer offene/geschlossene HorizonWindows liefern;
- v0.1-Kompatibilitaet ueber Adapter oder Re-Exports abdecken.

23 Referenz-Zustandsmaschine

Von	Nach	Bedingung
Idle	Loading	Input und RD vorhanden.
Loading	Canonicalizing	Dateien gelesen und Schema gueltig.
Canonicalizing	NullProjecting	Canonical bytes erzeugt.
NullProjecting	Scheduling	NullCenterObject erzeugt.
Scheduling	HorizonEvaluating	SchedulerState erzeugt.
HorizonEvaluating	Mounting	Horizon offen oder Policy erlaubt ProjectionOnly.
HorizonEvaluating	Holding	Horizon geschlossen und Finalisierung verlangt.
Mounting	Focusing	Alle ProjectionLayers gemountet.
Focusing	LoopbackTesting	TelescopeView bestimmt.
LoopbackTesting	Condensing	Loopback instabil.
LoopbackTesting	SeamEvaluating	Loopback stabil.
Condensing	SeamEvaluating	W/K/T/P-Zyklus abgeschlossen.
SeamEvaluating	Witnessing	Alle SeamReports erzeugt.
Witnessing	RealityCutting	Witness optional oder bestanden.
RealityCutting	QualityScoring	Pfade exzidiert/erhalten.
QualityScoring	ProjectionGating	QualityCascade berechnet.
ProjectionGating	ExclusionEvaluating	UPF-Gate bestanden.
ProjectionGating	Holding	UPF-Gate fehlgeschlagen.
ExclusionEvaluating	ExpansionEvaluating	Exclusion bestanden.
ExclusionEvaluating	Holding	Exclusion fehlgeschlagen.
ExpansionEvaluating	FinalizationEvaluating	Expansion bestanden.
ExpansionEvaluating	Holding	Expansion fehlgeschlagen.
FinalizationEvaluating	MatrixGating	Finalization bestanden.
FinalizationEvaluating	Holding	Finalization fehlgeschlagen.
MatrixGating	Finalized	HolisticMatrixGate bestanden.
MatrixGating	Holding	MatrixGate fehlgeschlagen.
Finalized	Certified	FinalizedEmission und Certificate erzeugt.
Holding	Certified	HoldReport und Certificate erzeugt.
Certified	Replayed	Replay erfolgreich.
jede Phase	Failed	Schema-, Determinismus-, Invarianz- oder Hashverletzung.

24 Schlussformel

Die v0.2-Bedingung lautet:

$$HGF(s, RD, k) = FinalizedEmission(s)$$

nur wenn $\delta(s)$ kanonisch $\wedge G_{UPF} = 1 \wedge G_{horizon} = 1 \wedge G_{exclusion} = 1 \wedge G_{expansion} = 1 \wedge G_{finalization} = 1 \wedge ReplayRecovery$

Andernfalls gilt:

$$HGF(s, RD, k) = Hold(s, diagnostics).$$